

HR & PAYROLL MANAGEMENT PLATFORM

Sprint 3 Developer Handbook

Implementation reference — Waves 1, 2 & 3

Companion to	Sprint 3 High-Level Design
Status	Delivered — all three waves shipped
Generated	September 9, 2026

Companion to `HR_Payroll_Sprint3_HLD.md` . Covers implementation detail for all three delivered waves — Wave 1 (careers page + candidate portal, calendar integration, polygon geofence, SMS/push channels, standalone reimbursement), Wave 2 (asset management, performance/KPI, learning management), Wave 3 (bank disbursement export, AI HR agent). Follows the same `schema → service → controller → routes → tests` module boundary and requirement-ID-in-comment convention as the Sprint 1/2 codebase.

1. Build Order (as shipped)

Wave 1 — five items, shallow dependencies, built in this order to keep each PR reviewable and each migration additive-only:

1. `GEO-10..13` (polygon geofence) — self-contained, extends an existing module.
2. `EXP-09..11` (standalone reimbursement) — self-contained, extends an existing module.
3. `NOTIF-08..13` (SMS/push channels) — self-contained, extends an existing module.
4. `CAREER-01..10` (careers page + candidate portal) — new module, introduces the candidate-auth pattern.
5. `CAL-01..07` (calendar integration) — depends on (4)'s structured `interviewers` field existing on `Interview` before it's meaningful to sync to Google Calendar.

Wave 2 — three items, product-decided order: `ASSET-*` → `PERF-*` → `LMS-*` . Assets first (simplest resource-scope shape to re-establish the pattern for this wave); Performance next (introduces `Employee.managerId` , reused implicitly by nothing in LMS but worth landing before the wave's most novel piece); LMS last.

Wave 3 — two items: `BANK-01` (bank disbursement export) before `AI-01..05` (AI HR agent), so the AI module could be built and tested against a codebase whose reusable list-function signatures (including the newly-added `Employee` bank fields) were already final.

2. New/Changed Backend Modules (cumulative)

backend/src/modules/geofence/	# existing – extended for polygon shape (Wave 1)
backend/src/modules/expenses/	# existing – extended for standalone reimburseme
backend/src/modules/notifications/	# existing – extended for SMS/push channels (Wav
backend/src/modules/recruitment/	# existing – extended for calendar integration (
backend/src/modules/careers/	# NEW (W1) – public careers listing + candidate
backend/src/modules/calendar/	# NEW (W1) – org calendar integration connect/ca
backend/src/modules/employees/	# existing – extended for managerId (W2) + bank
backend/src/modules/assets/	# NEW (W2) – asset register, assign/return
backend/src/modules/performance/	# NEW (W2) – review cycles + reviews
backend/src/modules/courses/	# NEW (W2) – course catalog + enrollments
backend/src/modules/payroll/	# existing – extended for disbursement CSV export
backend/src/modules/ai/	# NEW (W3) – AI HR assistant chat endpoint
backend/src/common/notifications/	# NEW (W1) – SmsProvider/PushProvider adapters
backend/src/common/auth/requireCandidateAuth.ts	# NEW (W1) – candidate JWT middleware
backend/src/common/crypto/tokenCrypto.ts	# NEW (W1) – AES-256-GCM helper for calendar tok
backend/src/common/csv/buildCsv.ts	# NEW (W3) – hand-rolled RFC 4180 CSV writer

3. Wave 1

3.1 Data Model Changes

Every change is additive (new nullable columns / new tables) except the `Interview.interviewerNames` → `interviewers` rename, which shipped with a one-time data migration (comma-split the free-text into `[{name, email: null}]`) rather than a breaking drop.

```

enum GeofenceShape { CIRCLE POLYGON }

model GeofenceZone {
  shape      GeofenceShape @default(CIRCLE)
  lat        Float?        // CIRCLE only
  lng        Float?        // CIRCLE only
  radiusMeters Int?        // CIRCLE only
  polygon     Json?        // POLYGON only – ordered [{lat,lng}], 3-50 points
}

enum DisbursementMethod { BANK_TRANSFER CHEQUE CASH OTHER }

model ExpenseClaim {
  disbursementMethod DisbursementMethod?
  disbursementReference String?
  disbursedAt        DateTime?
  disbursedBy        String? // User.id
}

model NotificationPreference {
  smsEnabled Boolean @default(true)
  pushEnabled Boolean @default(true)
}

model User { phoneNumber String? } // E.164, validated in employees/profile schema

model PushSubscription {
  id String @id @default(uuid())
  userId String
  endpoint String
  p256dh String
  auth String
  createdAt DateTime @default(now())
  @@unique([userId, endpoint])
}

model Organization {
  careersSlug String? @unique
  careersEnabled Boolean @default(false)
}

enum CandidatePortalStatus { ACTIVE INACTIVE }

model CandidatePortalAccount {
  id String @id @default(uuid())
  candidateId String @unique
  passwordHash String
  status CandidatePortalStatus @default(ACTIVE)
  lastLoginAt DateTime?
}

model CalendarIntegration {
  id String @id @default(uuid())
  organizationId String @unique
  provider String @default("GOOGLE")
  accessTokenEnc String
}

```

```

refreshTokenEnc String
expiresAt DateTime
calendarId String
connectedByUserId String
status String @default("ACTIVE")
}

model Interview {
  // interviewerNames String? - REMOVED, backfilled into `interviewers`
  interviewers Json // [{ name: string, email: string | null }]
  calendarEventId String?
}

```

3.2 GEO-10..13 — Polygon Geofence Zones

Files: `geofence.schema.ts` (Zod discriminated union: `CIRCLE` requires `lat / lng / radiusMeters`, `POLYGON` requires 3–50 `{lat,lng}` entries) · `geofence.service.ts` (`pointInPolygon(point, polygon)` via ray casting next to the existing `haversineMeters`; the attendance check-in path's `insideAnyZone` branches on `zone.shape`) · `frontend/src/features/geofence/` (new polygon-drawing form; adds `leaflet` + `react-leaflet` to `frontend/package.json`).

Tests (`geofence.test.ts`): point-in-polygon for convex/concave polygons and a boundary point (inclusive-boundary behavior defined explicitly); a zone list containing both shapes; existing circular-zone tests unmodified (regression guard).

Known issue fixed during implementation: the polygon/circle shape `<select>` in the frontend form used react-hook-form's default `shouldUnregister: false`, which left `radiusMeters` as a stale empty string from the unmounted circle fields when switching to polygon mode — it coerced to `0` and silently failed Zod's `.positive()` with no visible error (the failing field wasn't even rendered). Fixed by setting `shouldUnregister: true` on the form.

3.3 EXP-09..11 — Standalone Reimbursement

Files: `expenses.schema.ts` (`ReimburseInput`: `disbursementMethod`, `disbursementReference`, optional `disbursedAt` default now) · `expenses.service.ts` (`reimburseExpenseClaim`: guard `status === 'APPROVED'`, guard `payrollItemId === null`, set the four disbursement fields + status `REIMBURSED`, write `AuditLog`, emit the existing notification event) · `expenses.controller.ts` / `.routes.ts` — `POST /expenses/:id/reimburse`, `requireRole("HR_ADMIN", "SUPER_ADMIN")`.

Tests (`expenses.test.ts`): reimburse an approved claim with no payroll run in the period; reject reimbursing a DRAFT/SUBMITTED/REJECTED claim; reject reimbursing a claim that already has a `payrollItemId`; audit log row written.

3.4 NOTIF-08..13 — SMS/Push Channels

Files: `common/notifications/smsProvider.ts` (`SmsProvider` interface + `ConsoleSmsProvider`) · `common/notifications/pushProvider.ts` (`PushProvider` interface + `ConsolePushProvider`) · `notification.templates.ts` (`smsBody`, ≤160 chars, per event type) · `notification.queue.ts` (`processOneEvent` fans out to SMS/push after the email-stub call; per-channel try/catch so one

channel's failure never blocks another's) · `notifications.controller.ts` / `.schema.ts` (preferences PATCH extended) · `employees.*` (`phoneNumber` on the profile schema) · frontend service worker registration + push subscription flow, phone number field, SMS/push toggles.

Tests: SMS sent only when `phoneNumber` present and `smsEnabled` ; push sent only to active subscriptions; a provider throw doesn't prevent the event reaching `PROCESSED` ; NOTIF-04's no-sensitive-data rule holds for the new `smsBody` templates.

3.5 CAREER-01..10 — Public Careers Page & Candidate Portal

New module: `backend/src/modules/careers/` — `careers.routes.ts` , `.controller.ts` , `.service.ts` , `.schema.ts` , a field-filtered candidate-view serializer, `.test.ts` .

New middleware: `backend/src/common/auth/requireCandidateAuth.ts` — verify → re-read account status → attach `req.candidateAuth` , signed with `env.candidateJwtSecret` . A `CandidateAuthContext` type parallel to (never merged into) `AuthContext` .

Routes:

```
GET /api/v1/careers/:orgSlug/jobs # public
GET /api/v1/careers/:orgSlug/jobs/:id # public
POST /api/v1/careers/:orgSlug/register # public, rate-limited
POST /api/v1/careers/:orgSlug/login # public, rate-limited
GET /api/v1/candidate-portal/me # requireCandidateAuth
POST /api/v1/candidate-portal/jobs/:jobId/apply # requireCandidateAuth (resume upload)
GET /api/v1/candidate-portal/applications # requireCandidateAuth
GET /api/v1/candidate-portal/applications/:id # requireCandidateAuth
```

Env additions: `CANDIDATE_JWT_SECRET` (required in production, same fail-fast pattern as `JWT_SECRET`), `CANDIDATE_JWT_EXPIRES_IN` (default `8h`).

Tests (`careers.test.ts`): public listing returns `OPEN` postings only, only for orgs with `careersEnabled` , never another org's jobs for a wrong/missing slug; candidate JWT rejected by internal routes and vice versa; candidate can't see another candidate's applications; field filtering hides interviewer/feedback/salary data; REC-03 duplicate-open-application still enforced via the portal.

3.6 CAL-01..07 — Google Calendar Integration

Files: `common/crypto/tokenCrypto.ts` (AES-256-GCM `encryptToken` / `decryptToken` , key from `env.calendarTokenEncKey`) · `modules/calendar/` (new module: `calendar.controller.ts` , `.service.ts` , `.routes.ts` , `.test.ts` — `assertGoogleConfigured()` guards every action that needs Google credentials) · `recruitment.service.ts` (interview create/update/cancel calls the provider when `CalendarIntegration.status === 'ACTIVE'` , wrapped in try/catch so a Google API failure logs a warning without losing the interview record).

Routes (mounted at `/organization` , alongside `organizations.routes.ts`):

```
GET    /api/v1/organization/calendar-integration      # status, HR-only
GET    /api/v1/organization/calendar-integration/connect-url # HR-only
GET    /api/v1/organization/calendar-integration/callback # public (Google redirect t
DELETE /api/v1/organization/calendar-integration      # disconnect, HR-only
```

The OAuth `state` parameter is a signed JWT (`env.jwtSecret` , 10-minute expiry) carrying `{organizationId, userId}` — the callback is an unauthenticated top-level browser redirect from Google, so this is how it's correlated back to the initiating org/user without a session.

Env additions: `GOOGLE_CLIENT_ID` , `GOOGLE_CLIENT_SECRET` , `GOOGLE_REDIRECT_URI` , `CALENDAR_TOKEN_ENC_KEY` (32-byte hex, `openssl rand -hex 32`).

Data migration: one-time SQL backfill converting `interviewerNames` (`"Alice, Bob"`) into `interviewers` (`[{name:"Alice",email:null}, {name:"Bob",email:null}]`) — emails are unknown for historical rows and stay `null` .

Tests: interview scheduling with no `CalendarIntegration` connected behaves exactly like Sprint 2 (no error, no event); config-missing error path (no Google Cloud project exists for this deployment, so `.env.test` leaves these unset — same "optional, degrades gracefully" treatment as VAPID keys); OAuth `state` validation (fails before ever calling Google, so it's testable without one); token encryption round-trips.

4. Wave 2

4.1 Data Model Changes

```
model Employee { managerId String? } // self-relation

enum AssetCategory { LAPTOP MONITOR PHONE PERIPHERAL FURNITURE OTHER }
enum AssetStatus { AVAILABLE ASSIGNED IN_REPAIR RETIRED }

model Asset {
  id String @id @default(uuid())
  organizationId String
  assetTag String
  category AssetCategory @default(OTHER)
  status AssetStatus @default(AVAILABLE)
  currentEmployeeId String?
  @@unique([organizationId, assetTag])
}

model AssetAssignment {
  assetId String
  employeeId String
  assignedAt DateTime @default(now())
  assignedBy String // User.id, unlinked (same pattern as ExpenseClaim.approvedBy)
  returnedAt DateTime?
  returnedBy String?
}

enum ReviewCycleStatus { DRAFT OPEN CLOSED }

model PerformanceReviewCycle {
  id String @id @default(uuid())
  organizationId String
  periodStart DateTime @db.Date
  periodEnd DateTime @db.Date
  status ReviewCycleStatus @default(DRAFT)
}

model PerformanceReview {
  cycleId String
  employeeId String
  goals String?
  selfRating Int?
  selfComments String?
  selfSubmittedAt DateTime?
  managerRating Int?
  managerComments String?
  managerSubmittedAt DateTime?
  managerUserId String? // actual submitter – may differ from employee.managerId
  @@unique([cycleId, employeeId])
}

enum CourseStatus { ACTIVE ARCHIVED }
```

```

model Course {
  id String @id @default(uuid())
  organizationId String
  externalUrl String
  category String?
  mandatory Boolean @default(false)
  status CourseStatus @default(ACTIVE)
}

model CourseEnrollment {
  courseId String
  employeeId String
  enrolledAt DateTime @default(now())
  completedAt DateTime? // set ⇒ COMPLETED, unset ⇒ ENROLLED – no separate status column
  @@unique([courseId, employeeId])
}

```

4.2 ASSET-01..08 — Asset Management

Files: `assets.schema.ts`, `.service.ts`, `.controller.ts`, `.routes.ts`, `.test.ts`. `listAssets` uses the force-scope pattern (HLD §3): non-HR requesters are restricted to `currentEmployeeId === their own employee.id` regardless of any `employeeId` query filter. `updateAsset` explicitly rejects a direct PATCH to `status: ASSIGNED` with `AppError.conflict("ASSET_CURRENTLY_ASSIGNED", ...)` — the only way into `ASSIGNED` is `assignAsset`, and the only way out is `returnAsset` (or a PATCH to `IN_REPAIR` / `RETIRED` after a return), keeping `Asset.status` / `Asset.currentEmployeeId` / the open `AssetAssignment` row from ever going out of sync.

Routes:

```

GET    /api/v1/assets           # requireAuth (force-scoped)
POST   /api/v1/assets         # HR-only
PATCH /api/v1/assets/:id      # HR-only
POST   /api/v1/assets/:id/assign # HR-only – requires AVAILABLE
POST   /api/v1/assets/:id/return # HR-only – closes the open AssetAssignment row
GET    /api/v1/assets/:id/assignments # HR-only – full assignment history for one asset

```

Frontend: `features/assets/` — `AssetsPage.tsx` (role switch), `HrAssetsAdmin.tsx` (register CRUD, assign/return), `MyAssets.tsx` (employee's own currently-assigned list).

Tests: assign requires `AVAILABLE`; assign a currently-`ASSIGNED` asset is rejected; return clears `currentEmployeeId` and closes the assignment row; a direct PATCH to `ASSIGNED` is rejected; non-HR `GET /assets` never returns another employee's assets regardless of an `employeeId` query param.

4.3 PERF-01..07 — Performance / KPI Reviews

Files: `performance.schema.ts`, `.service.ts`, `.controller.ts`, `.routes.ts`, `.test.ts`. `openReviewCycle` bulk-creates a `PerformanceReview` row for every then-`ACTIVE` employee in the org. `resourceScopeWhere` (internal helper) returns "own review OR a direct report's" for non-HR requesters

— the reason `list/get/self/manager` routes are `requireAuth` -only rather than role-gated: an employee submitting their own self-review and a manager submitting a report's manager-section are both legitimate callers of the same endpoints.

Routes:

```
GET    /api/v1/performance/cycles          # HR-only
POST   /api/v1/performance/cycles          # HR-only
POST   /api/v1/performance/cycles/:id/open  # HR-only – bulk-creates reviews
POST   /api/v1/performance/cycles/:id/close # HR-only
GET    /api/v1/performance/reviews        # requireAuth, resource-scoped
GET    /api/v1/performance/reviews/:id     # requireAuth, resource-scoped
PATCH /api/v1/performance/reviews/:id/self  # requireAuth
PATCH /api/v1/performance/reviews/:id/manager # requireAuth
```

Frontend: `features/performance/` — `PerformancePage.tsx`, `ReviewDetailPage.tsx` (self/manager submission forms, status derived client-side from the same timestamp fields the backend uses).

Tests: opening a cycle creates a review per active employee (and skips inactive ones); self/manager submission each set their own timestamp+rating+comments independently; a non-manager, non-HR employee cannot read or submit another employee's review; closing a cycle blocks further submissions regardless of role.

4.4 LMS-01..07 — Learning Management

Files: `courses.schema.ts`, `.service.ts`, `.controller.ts`, `.routes.ts`, `.test.ts`.

`listEnrollments` uses the same force-scope shape as assets. `completeCourse` is self-reported — any enrolled employee can mark their own enrollment complete, no manager sign-off.

Routes:

```
GET    /api/v1/courses                  # requireAuth – full catalog
POST   /api/v1/courses                  # HR-only
PATCH /api/v1/courses/:id              # HR-only
POST   /api/v1/courses/:id/enroll       # requireAuth – self-enroll
POST   /api/v1/courses/:id/enroll-all-active # HR-only – bulk-enroll every active employee
POST   /api/v1/courses/:id/complete     # requireAuth – self-reported
GET    /api/v1/courses/enrollments      # requireAuth, force-scoped
```

Frontend: `features/courses/` — `CoursesPage.tsx` (role switch), `CourseCatalog.tsx` (browse/enroll/complete), `HrCoursesAdmin.tsx` (catalog CRUD + bulk-enroll).

Tests: self-enroll creates one enrollment per (course, employee) pair (`@unique` enforced); bulk-enroll skips employees already enrolled; complete sets `completedAt`; a non-HR requester's enrollment list never includes another employee's rows regardless of an `employeeId` query param.

5. Wave 3

5.1 Data Model Changes

```
model Employee {
  bankName      String?
  bankAccountName String?
  bankAccountNumber String?
}
```

No new tables in Wave 3 — both items are additive columns / a new stateless endpoint.

5.2 BANK-01 — Bank Disbursement File Export

Files: `common/csv/buildCsv.ts` (+ `.test.ts`) — a ~15-line RFC 4180 writer (`escapeCell` quotes/doubles embedded quotes, rows joined with `\r\n`); no CSV library dependency was added since the shape needed is a header row plus flat scalar rows. `payroll.service.ts` — `generatePayrollDisbursementCsv(organizationId, runId)`: requires run status `APPROVED` or `PAID` (else `AppError.conflict("PAYROLL_RUN_NOT_APPROVED", ...)`), one row per `PayrollItem` with the employee's bank fields (blank if unset) and `net pay`. `expenses.service.ts` — `generateExpenseDisbursementCsv(organizationId)`: every `APPROVED` claim with `payrollItemId: null` (i.e. not already paid through a payroll run) — read-only, does not itself mark anything `REIMBURSED`.

Routes:

```
GET /api/v1/payroll/runs/:id/disbursement-file # HR-only, text/csv attachment
GET /api/v1/expenses/disbursement-file        # HR-only, text/csv attachment
```

Frontend: `features/payroll/PayrollRunDetailPage.tsx` — "Download disbursement file" button, shown once a run is `APPROVED` / `PAID`. `features/expenses/HrExpenseApprovals.tsx` — download button + explanatory copy that this is an export, not a reimbursement action. `features/employees/EmployeeDetailPage.tsx` — new `BankDetailsSection`, plain controlled inputs, save disabled unless dirty. Both download actions use the blob-based authenticated download pattern (`responseType: "blob"`, `window.URL.createObjectURL`) — a plain `<a href>` can't carry the `Authorization` header this endpoint requires.

Tests: rejecting a disbursement-file request for a DRAFT/CALCULATED run; correct CSV content and header row for both endpoints; RBAC 403 for a non-HR caller; blank bank fields render as empty CSV cells rather than erroring.

5.3 AI-01..05 — AI HR Agent

New module: `backend/src/modules/ai/` — `ai.schema.ts` (`chatRequestSchema`: `message` 1–4000 chars, `history` array of `{role, content}` capped at 20 entries) · `ai.service.ts` (`buildEmployeeSnapshot` + `chatWithAssistant`) · `ai.controller.ts` · `ai.routes.ts` · `ai.test.ts`.

`buildEmployeeSnapshot(organizationId, userId)` resolves the caller's own `Employee` row, then `Promise.all`s calls into the *existing* list functions from six other modules — `listLeaveBalances`, `listAttendance`, `listPayslips`, `listExpenseClaims`, `listAssets`, `listReviews`, `listEnrollments` — each called with `requester: { userId, role: "EMPLOYEE" as const }`, a synthetic role that ignores the caller's actual role entirely. This is the same trick recorded in HLD §3/§4.10: it's what makes it structurally impossible for the assistant to return org-wide data even when the real caller is HR_ADMIN/SUPER_ADMIN.

`chatWithAssistant` guards with `assertConfigured()` (mirrors `calendar.service.ts`'s `assertGoogleConfigured()` — throws `AppError.badRequest("AI_ASSISTANT_NOT_CONFIGURED", ...)` if `env.anthropicApiKey` is unset), builds a system prompt embedding the JSON snapshot plus explicit "answer only from this data, never fabricate, no actions" instructions, then makes one `anthropic.messages.create()` call (`env.anthropicModel`, default `claude-sonnet-5`, `max_tokens: 1024`) with the client-supplied history spliced in ahead of the new message.

Route:

```
POST /api/v1/ai/chat # requireAuth, aiChatRateLimiter (10/min, keyed per user)
```

`aiChatRateLimiter` (new, `common/middleware/rateLimiter.ts`) is keyed on `req.auth.userId` rather than IP — the existing `loginRateLimiter` is IP-based, which would let one office NAT/shared IP throttle every employee behind it on an endpoint that costs real money per call.

Env additions:

```
ANTHROPIC_API_KEY= # optional — unset ⇒ AI_ASSISTANT_NOT_CONFIGURED, app still start
ANTHROPIC_MODEL=claude-sonnet-5
```

Frontend: `features/assistant/` — `AssistantPage.tsx` (chat UI, message list held in local component state only, `history` resent each call capped client-side to the last 20 messages), `api.ts` (`sendMessage`), `types.ts`. Routed at `/assistant`, added to the sidebar nav alongside the other self-service items (Attendance, Leave, Payslips, Assets, Performance, Learning).

Tests (`ai.test.ts`): unauthenticated request rejected (401); config-missing error path — no `ANTHROPIC_API_KEY` in `.env.test`, same "optional, degrades gracefully" treatment as Google Calendar credentials, so this is reachable without ever calling the real Claude API; request-body validation (empty message rejected). RBAC/self-scope behavior against a live snapshot would need either a real API key or mocking the Anthropic client — not exercised in CI; flagged as a manual smoke-test item instead (§6).

6. Rollout Checklist

- ☐ Wave 1's five migrations, Wave 2's asset/performance/course migrations, and Wave 3's `Employee` bank-field migration all applied in the order listed in §1, each independently deployable.

- ☐ `.env.example` documents every new var across all three waves; `CANDIDATE_JWT_SECRET` and `CALENDAR_TOKEN_ENC_KEY` are required-in-production (same fail-fast pattern as `JWT_SECRET`); `ANTHROPIC_API_KEY`, `GOOGLE_CLIENT_ID` / `SECRET`, and `VAPID_*` remain optional, degrade-gracefully vars.
- ☐ `docker-compose.yml` — no new services required across any wave (SMS/push are stubs; Calendar and the AI assistant are outbound HTTPS calls, not new containers).
- ☐ Existing Sprint 1/2 test suites pass unmodified (regression guard).
- ☐ Manual smoke test: circular geofence check-in still works; a payroll-cycle expense reimbursement still works; an HR-entered (non-portal) candidate application still works; assigning then returning an asset leaves it `AVAILABLE`; opening then closing a review cycle blocks further submissions; the AI assistant chat UI renders and surfaces `AI_ASSISTANT_NOT_CONFIGURED` cleanly when no key is set, and (once a real key is configured in a target environment) returns a grounded answer referencing the asking employee's actual data.